

EXPERIMENT - 7

Build a contact form with validation using Angular 16 reactive forms (intro-level Angular usage).

OBJECTIVE OF THE EXPERIMENT

To develop a Contact Form using Angular 16 Reactive Forms with built-in form validation. The experiment enables students to create a responsive form, validate user inputs, display validation messages, and understand the basics of Angular reactive forms.

OUTCOME OF THE EXPERIMENT

After completing this experiment, students will be able to:

1. Understand the concept of Angular Reactive Forms.
2. Create a contact form using FormGroup and FormControl.
3. Apply built-in validators such as Required, Email, and Minimum Length.
4. Display validation error messages dynamically.
5. Handle form submission using Angular.
6. Improve user experience through client-side validation.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB (Minimum)
- Hard Disk: 2 GB Free Space

Software Requirements

- Operating System: Windows 10/11 / Linux / macOS
- Node.js (Latest LTS)
- Angular CLI 16
- Visual Studio Code
- Google Chrome or Microsoft Edge
- npm (Node Package Manager)

ALGORITHM

1. Start.
2. Create a new Angular project.
3. Import the ReactiveFormsModule.
4. Create a Contact Component.
5. Define a FormGroup with the required controls.
6. Apply validators to each control.
7. Design the HTML form using Angular directives.
8. Display validation messages for invalid inputs.
9. Check whether the form is valid.
10. If valid, display a success message.
11. Otherwise, display validation errors.
12. Stop.

PSEUDOCODE:

BEGIN

Create ContactForm

Add fields

 Name

 Email

 Subject

 Message

Apply Validators

 Name -> Required

 Email -> Required + Email

 Subject -> Required

 Message -> Required + Minimum Length

WHEN Submit Button is Clicked

 IF Form is Valid THEN

 Display "Form Submitted Successfully"

 ELSE

Display Validation Messages

ENDIF

END

app.module.ts

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { ReactiveFormsModule } from '@angular/forms';
```

```
import { AppComponent } from './app.component';
```

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [  
    BrowserModule,  
    ReactiveFormsModule  
  ],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

app.component.ts

```
import { Component } from '@angular/core';  
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
@Component({  
  selector: 'app-root',
```

```

    templateUrl: './app.component.html'
  })
  export class AppComponent {

    contactForm = new FormGroup({
      name: new FormControl("", Validators.required),
      email: new FormControl("", [
        Validators.required,
        Validators.email
      ]),
      subject: new FormControl("", Validators.required),
      message: new FormControl("", [
        Validators.required,
        Validators.minLength(10)
      ])
    });

    onSubmit() {
      if (this.contactForm.valid) {
        alert("Form Submitted Successfully");
        console.log(this.contactForm.value);
      }
    }
  }
}

```

App.component.html

```

<h2>Contact Form</h2>

<form [formGroup]="contactForm" (ngSubmit)="onSubmit()">

  <label>Name</label><br>
  <input type="text" formControlName="name">

  <div *ngIf="contactForm.controls.name.invalid && contactForm.controls.name.touched">
    Name is required.
  </div>

  <br><br>

```

```
<label>Email</label><br>
<input type="email" formControlName="email">

<div *ngIf="contactForm.controls.email.invalid && contactForm.controls.email.touched">
  Enter a valid email.
</div>

<br><br>

<label>Subject</label><br>
<input type="text" formControlName="subject">

<div *ngIf="contactForm.controls.subject.invalid && contactForm.controls.subject.touched">
  Subject is required.
</div>

<br><br>

<label>Message</label><br>

<textarea rows="5" formControlName="message"></textarea>

<div *ngIf="contactForm.controls.message.invalid && contactForm.controls.message.touched">
  Message should contain at least 10 characters.
</div>

<br><br>

<button type="submit" [disabled]="contactForm.invalid">
  Submit
</button>

</form>
```

OUTPUT

Contact Form

Name : _____

Email : _____

Subject : _____

Message :

[Submit]

If fields are empty:

Name is required.
Enter a valid email.
Subject is required.
Message should contain at least 10 characters.

If valid:

Form Submitted Successfully

Pre-Lab Questions

1. What are Reactive Forms in Angular, and why are they used?
2. What is the purpose of the ReactiveFormsModule in Angular?
3. What is the difference between a FormGroup and a FormControl?
4. Name any four built-in validators available in Angular Reactive Forms.
5. What is the purpose of the FormControlName directive?

Pre-Viva Questions

1. What is the difference between Template-driven Forms and Reactive Forms in Angular?
2. What is the role of the `Validators.required` validator?
3. How do you check whether a Reactive Form is valid before submitting it?
4. What is the purpose of the `(ngSubmit)` event in Angular forms?
5. Why is form validation important in web applications?